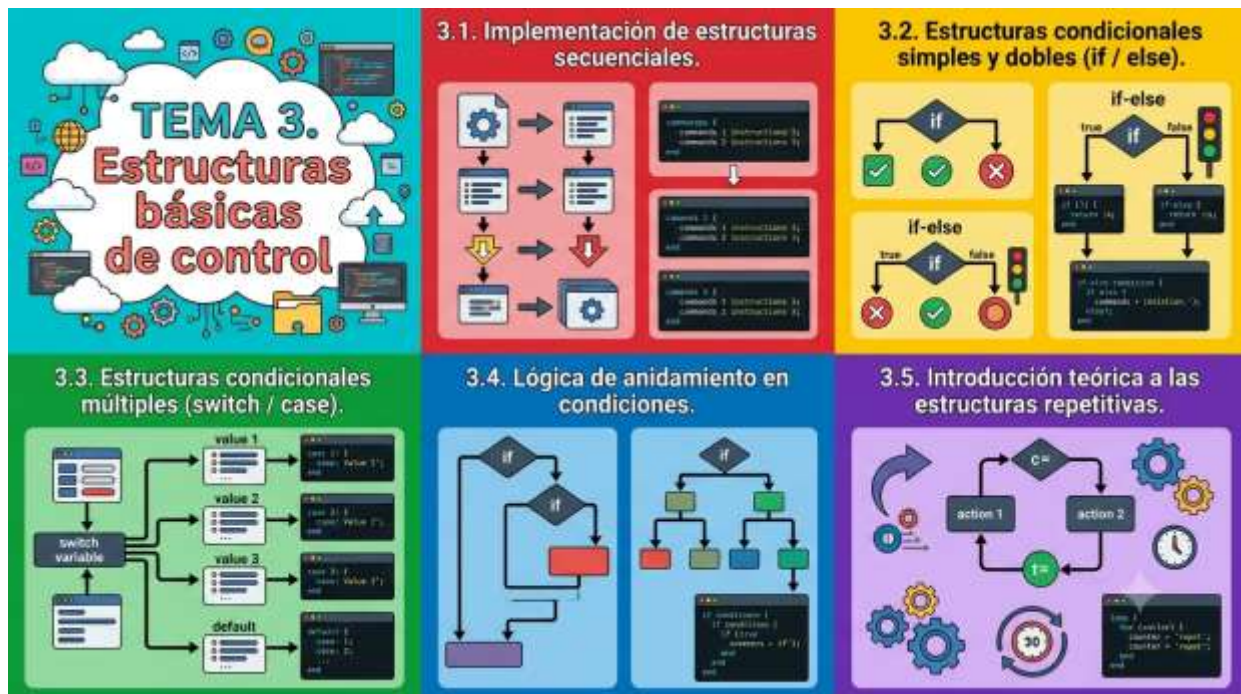


TEMA N° 3



TEMA 3 ESTRUCTURAS BÁSICAS DE CONTROL



EN ESTE TEMA SERÁS CAPAZ DE...

1. **Comprender y estructurar el flujo de ejecución de un programa**, identificando con total claridad cómo se procesan las instrucciones secuenciales, cómo se bifurcan los caminos mediante condiciones y cómo se preparan los fundamentos para la automatización cíclica de procesos en el desarrollo de software.
2. **Diseñar e implementar lógica condicional avanzada (simple, doble, múltiple y anidada)** para resolver problemas computacionales reales y automatizar la toma de decisiones dentro de sistemas de software, garantizando un código limpio, legible y eficiente.



3. **Modelar algoritmos robustos aplicados a necesidades del entorno sociocomunitario**, transformando requisitos informales en estructuras de código lógicas, evaluando el impacto ético de las decisiones automatizadas y preparándote para ejercer como un Técnico capacitado ante los desafíos del mercado tecnológico actual.



PRÁCTICA



PREGUNTA DETONANTE

Si observas las actividades cotidianas de los comercios en el Barrio Central Fátima, el movimiento de la estación de tren en el Barrio 24 de Septiembre o la organización de un campeonato en el coliseo municipal del Barrio 27 de Mayo, notarás que todo sigue un orden: algunas tareas se hacen una tras otra, otras dependen de una condición clave (como "si hay energía eléctrica, entonces encender el equipo"), y otras se repiten de forma constante. En nuestro día a día tomamos decisiones continuamente basándonos en variables concretas. Si el cerebro humano procesa estas situaciones de manera natural, ¿cómo podemos transferir este razonamiento a una computadora para que aprenda a tomar decisiones autónomas y resuelva problemas críticos en nuestra comunidad?

PLANTEAMIENTO DE LA SITUACIÓN PROBLEMÁTICA

El CEA Herman Ortiz Camargo, ubicado en el pujante Barrio Saó a cinco cuadras de la población de Pailón, se encuentra estrenando su flamante módulo educativo propio, el cual cuenta con laboratorios equipados con tecnología moderna. La dirección del establecimiento, en coordinación con las juntas vecinales de los barrios aledaños como María Belén, San Antonio y Jardines de Pailón, ha identificado una problemática urgente: el crecimiento demográfico y comercial de Pailón ha saturado la gestión manual de los servicios locales.

Imagina que eres el Técnico en Sistemas Informáticos encargado de diseñar el prototipo de software para la gestión de tres escenarios críticos del municipio:

1. **Control de Peaje y Carga en el Barrio 24 de Septiembre:** Los trenes y camiones de carga que cruzan la estación necesitan un sistema automatizado que calcule el impuesto de



- rodaje básico, aplique penalizaciones si exceden el peso permitido y derive los registros a la oficina central de recaudaciones del Barrio Central Fátima.
2. **Cronograma del Coliseo en el Barrio 27 de Mayo:** La alcaldía requiere clasificar automáticamente las solicitudes de uso del coliseo municipal (Fútbol de salón, Básquetbol, Voleibol o Eventos Culturales), asignando un costo de limpieza diferenciado y un horario específico según la categoría elegida.
 3. **Monitoreo de Seguridad en el Módulo Educativo (Barrio Saó):** Para resguardar las computadoras del nuevo laboratorio del CEA, se necesita un sistema de acceso digital que valide la identidad del usuario, verifique si pertenece al turno correspondiente (mañana, tarde o noche) y evalúe si cuenta con autorización especial del docente para ingresar fuera de horario.

Hasta ahora, los programas que hemos desarrollado se ejecutan línea por línea, de arriba hacia abajo, sin desviarse jamás. Si el sistema de seguridad lee una tarjeta y el usuario no pertenece al CEA, el programa actual se colgaría o procesaría el acceso de todos modos, exponiendo los laboratorios a vulnerabilidades. Como Técnico, te enfrentas al reto de romper la linealidad del código e introducir bifurcaciones inteligentes que controlen el comportamiento del software de acuerdo con la realidad de los hechos en Pailón.



TEORÍA



3.1. IMPLEMENTACIÓN DE ESTRUCTURAS SECUENCIALES

La estructura secuencial es la forma más simple y natural en la que se ejecutan las instrucciones de un programa informático. En este modelo, las acciones se procesan de manera lineal, una a continuación de la otra, respetando estrictamente el orden en el que fueron escritas por el desarrollador de software. No existen saltos, desvíos ni repeticiones; cada línea de código se ejecuta exactamente una vez.

En el contexto de la programación, el flujo secuencial responde al esquema clásico de: **Entrada de datos** $\rightarrow$ **Procesamiento** $\rightarrow$ **Salida de resultados**.



Para comprenderlo con un caso práctico de nuestro municipio, supongamos que una tienda comercial del Barrio Central Fátima requiere calcular el total de una venta aplicando el impuesto municipal correspondiente al comercio formal. El flujo secuencial se implementa recolectando el precio del producto, sumando el impuesto y mostrando el monto final en la pantalla de la consola.

A continuación, analizamos la implementación en código Python para resolver esta necesidad de cálculo directo:

Python

```

# Programa para el cálculo automatizado de ventas en el Barrio Central Fátima
# Este programa demuestra el flujo estrictamente secuencial de la información
# Paso 1: Entrada de datos (Se solicita el costo del artículo de papelería o
# abarrotes)
precio_base = float(input("
Ingrese el precio base del producto (Bs.): ")) # Captura el dato y lo convierte
a número decimal
# Paso 2: Procesamiento de datos (Se realizan las operaciones aritméticas de
# forma lineal)
tasa_impuesto = 0.13 # Define la constante del impuesto del 13% aplicable al
comercio
monto_impuesto = precio_base * tasa_impuesto # Calcula el valor neto del
impuesto a pagar
precio_total = precio_base + monto_impuesto # Suma el precio
base más el impuesto calculado
# Paso 3: Salida de resultados (Muestra la información procesada al usuario en

```



```
La terminal)
print("
--- RESUMEN DE COMPRA COMERCIAL -
--") # Imprime un encabezado estético de separación
print("
Precio original: Bs.", precio_base) # Muestra el valor que introdujo el
usuarioprint("
Impuesto aplicado (13%): Bs.", monto_impuesto) # Muestra el desglose del
impuesto municipalprint("
Total neto a cancelar: Bs.", precio_total) # Muestra el resultado final de la
ejecución secuencial
```

Como se aprecia en el código, el intérprete de Python no puede calcular el precio_total sin antes haber calculado el monto_impuesto, y este último requiere obligatoriamente la existencia de la variable precio_base. Cada instrucción es un eslabón dependiente del anterior.

3.2. ESTRUCTURAS CONDICIONALES SIMPLES Y DOBLES (IF / ELSE)

En la vida real, las secuencias rígidas son insuficientes. Los sistemas informáticos adquieren verdadera utilidad cuando son capaces de evaluar situaciones y tomar caminos distintos. Las estructuras condicionales permiten alterar el flujo secuencial del programa basándose en el resultado de una **expresión booleana** (una condición que solo puede ser verdadera o falsa).

La **condición simple (if)** evalúa una expresión. Si esta es verdadera (True), ejecuta un bloque de código específico; si es falsa (False), ignora dicho bloque y continúa con la instrucción inmediatamente posterior a la estructura condicional.

Por otro lado, la **condición doble (if / else)** ofrece dos caminos alternativos y excluyentes. Si la condición matemática o lógica es verdadera, se ejecuta el bloque de código dentro del if. Si la condición es falsa, el programa ignora el primer bloque y ejecuta de forma obligatoria el bloque contenido dentro del else.



Para ejemplificar esto en nuestra región, trasladémonos a la estación de carga del Barrio 24 de Septiembre. Imaginemos un sistema de pesaje automatizado que controle el ingreso de camiones de alto tonelaje que transportan grano desde las comunidades rurales hacia los silos de Pailón. Si el peso del camión supera las 25 toneladas, el sistema debe emitir una alerta de penalización por sobrepeso; de lo contrario, debe aprobar el tránsito libremente.

Observemos con detenimiento cómo se traduce este algoritmo a código limpio y eficiente:

Python

```

# Sistema de Control de Tránsito de Carga - Estación Barrio 24 de Septiembre
# Uso de estructura condicional doble para evaluar el tonelaje de los vehículos
# Captura el peso registrado por la báscula industrial del punto de control
peso_camion = float(input("
Ingrese el peso registrado del camión (Toneladas): "))
# Definición del límite legal permitido para las carreteras del
municipio
limite_permitido = 25.0 # Expresado en toneladas métricas
# Evaluación lógica mediante la estructura condicional doble
if peso_camion > limite_permitido:
    # Bloque Verdadero: Se ejecuta únicamente si el peso es estrictamente mayor a 25
    print("\n[ALERTA - SOBREPESO DETECTADO]") # Alerta visual para el operador
    exceso = peso_camion - limite_permitido # Calcula las toneladas excedentes
    print("El camión excede el límite por:", exceso, "toneladas.")
# Informa la gravedad de la falta
print("Estado: RETENIDO. Diríjase a la ventanilla del Barrio Central Fátima para la multa.")
  
```



```
else:    # Bloque Falso: Se ejecuta si el peso es menor o exactamente igual a 25
toneladas    print("\n[TRÁNSITO AUTORIZADO]") # Mensaje de confirmación
constructivo    print("
El vehículo se encuentra dentro del rango de peso reglamentario.") # Explicación
clara    print("
Estado: APROBADO. Puede continuar su ruta hacia los silos de almacenamiento.")
# Instrucción secuencial fuera de la condicional (se ejecuta siempre sin
importar el peso)
print("
--- Proceso de pesaje concluido correctamente -
--")
```

3.3. ESTRUCTURAS CONDICIONALES MÚLTIPLES (SWITCH / CASE)

Cuando un algoritmo se enfrenta a múltiples opciones excluyentes basadas en el valor de una sola variable, el uso de estructuras condicionales simples o dobles encadenadas puede volver el código engorroso, propenso a errores y difícil de mantener. Para solucionar esto, los lenguajes de programación implementan las estructuras condicionales múltiples, tradicionalmente conocidas como switch / case.

En Python moderno (versiones 3.10 en adelante), esta estructura se implementa de manera elegante mediante la instrucción match / case. Esta estructura compara el valor de una variable con diferentes patrones predefinidos. Cuando halla una coincidencia, ejecuta exclusivamente el bloque de código asociado a ese caso (case). Si el valor no coincide con ninguno de los patrones establecidos, se recurre a un caso por defecto (representado por un guion bajo case _), el cual actúa como un salvaguarda para capturar valores inesperados o erróneos.

Ubiquémonos ahora en el Barrio 27 de Mayo, específicamente en el sistema de reservas informáticas para las actividades deportivas y culturales que acontecen en el coliseo municipal. La alcaldía maneja una tarifa diferenciada de mantenimiento y limpieza según el tipo de disciplina que solicita el espacio. El sistema requiere procesar la opción seleccionada por el operario y asignar el costo de manera automática.

Analicemos la claridad con la que se redacta este proceso mediante condicionales múltiples:

Python

```
# Sistema de Asignación de Presupuesto e Infraestructura - Coliseo Barrio 27 de Mayo
```



```
# Implementación de condicionales múltiples para la optimización de
opcionesprint("
--- MENÚ DE RESERVAS DEL COLISEO MUNICIPAL -
--")
print("
A. Fútbol de Salón")
print("
B. Básquetbol")
print("
C. Voleibol")
print("
D. Evento Cultural / Folclórico")
# El usuario ingresa la letra correspondiente a la actividad que planea
realizarcodigo_actividad = input("
Seleccione el código de la actividad (A-D): ").upper() # Convierte a mayúsculas
para evitar fallas
# Aplicación de la estructura condicional múltiple moderna (match/case)
match codigo_actividad: case "
A": # Bloque ejecutado si la variable contiene exactamente "
A" disciplina = "
Fútbol de Salón" # Asigna el nombre de la disciplina costo_limpieza = 150
# Define el costo en Bolivianos para esta actividad intensa case "
B": # Bloque ejecutado si la variable contiene exactamente "
B" disciplina = "
Básquetbol" # Asigna el nombre de la disciplina costo_limpieza = 100 #
Costo asignado para el uso de tableros y canchas case "
C": # Bloque ejecutado si la variable contiene exactamente "
C" disciplina = "
Voleibol" # Asigna el nombre de la disciplina costo_limpieza = 80 # Costo
de mantenimiento para la red e iluminación case "
D": # Bloque ejecutado si la variable contiene exactamente "
D" disciplina = "
Evento Cultural / Acto de Unidades Educativas" costo_limpieza = 250 #
Mayor costo debido al despliegue de graderías y limpieza masiva case _:
# Caso por defecto: Se ejecuta si el usuario introduce cualquier otra letra o
número no válido disciplina = "
Desconocida / Opción Inválida" # Clasifica el error para evitar colapsos
costo_limpieza = 0 # No se asigna costo al no haber una reserva real
# Salida estructurada de la información recopilada por el programaprint("\n
--- DETALLE DEL REGISTRO DE INFRAESTRUCTURA -
--")
print("
Actividad catalogada:", disciplina) # Muestra la disciplina determinadaif
costo_limpieza > 0: # Condicional simple para verificar si el trámite es
válido y mostrar el canon print("
Costo de mantenimiento y limpieza obligatoria: Bs.", costo_limpieza)
else: # Informa al operador que debe reiniciar el software por ingresar un
parámetro erróneo print("
Por favor, seleccione una opción válida del menú e intente de nuevo.")
```




3.4. LÓGICA DE ANIDAMIENTO EN CONDICIONES

La lógica de anidamiento se presenta cuando una estructura condicional se encuentra colocada dentro del bloque de instrucciones de otra estructura condicional. En términos sencillos, es una "condición dentro de otra condición". Esto permite al Técnico modelar árboles de decisión sumamente complejos y precisos, donde para llegar a evaluar un requisito secundario, se debió haber aprobado obligatoriamente un filtro primario.

Aunque el anidamiento es una herramienta sumamente poderosa, es vital que el Técnico cuide con esmero la **indentación** (los espacios en blanco al inicio de las líneas de código). La indentación le indica al lenguaje de programación qué líneas pertenecen a qué nivel de la condición. Un error en los espacios provocará fallas lógicas devastadoras en el software o detendrá su ejecución por completo.



Para visualizar este concepto en un entorno de alta fidelidad con las tareas del Técnico, dirijámonos al nuevo y flamante módulo educativo del CEA Herman Ortiz Camargo en el Barrio Saó. El laboratorio de Sistemas Informáticos cuenta con un control de acceso lógico para resguardar los equipos informáticos de última generación. El software de control debe evaluar primero si la tarjeta inteligente pertenece a un estudiante activo del establecimiento; si pasa esta



prueba, el sistema debe evaluar el turno (Mañana o Noche) para permitir el acceso al laboratorio de desarrollo web según el horario establecido.

Examinemos detalladamente el diseño de este sistema mediante condiciones anidadas:

Python

```
# Sistema Inteligente de Control de Accesos - Laboratorio CEA Herman Ortiz
Camargo (Barrio Saó)
# Demostración práctica de lógica de anidamiento y dependencia
jerárquicaprint("=== SENSOR DE CONTROL DE ACCESO DIGITAL ===")
# Simulación de lectura de credenciales académicas
institucionaleses_estudiante_activo = input("¿El usuario está registrado en el
sistema del CEA? (si/no): ").lower()
# Condición Principal (Filtro de seguridad de primer nivel)
if es_estudiante_activo == "
si":    print("
Acceso de identidad: VALIDADO.") # El usuario pertenece al CEA          #
Condición Anidada (Filtro de seguridad de segundo nivel, solo se evalúa si la
primera fue '
si')    turno_solicitado = input("
Ingrese el turno actual de clases (mañana/noche): ").lower()          if
turno_solicitado == "
mañana":    # Bloque de operaciones exclusivo para el horario matutino
print("
Horario verificado: Turno Mañana.")          print("
Resultado: PUERTA DESBLOCKED. Bienvenido al laboratorio de Redes.")      elif
turno_solicitado == "
noche":    # Bloque de operaciones exclusivo para el horario nocturno
print("
Horario verificado: Turno Noche.")          print("
Resultado: PUERTA DESBLOCKED. Bienvenido al laboratorio de Programación.")
else:    # Captura variaciones de texto o turnos inexistentes          print("
Turno no reconocido por el sistema de administración.")          print("
Resultado: ACCESO DENEGADO temporalmente por inconsistencia horaria.")
else:    # Se ejecuta de inmediato si el usuario contestó '
no
' en la primera pregunta          print("
Acceso de identidad: RECHAZADO.")          print("
Alerta: El usuario no figura en la base de datos de Sistemas Informáticos del
CEA.")          print("
Resultado: PUERTA CERRADA. Se ha enviado una alerta al regente de turno.")
print("=====")
```



3.5. INTRODUCCIÓN TEÓRICA A LAS ESTRUCTURAS REPETITIVAS

Hasta este punto del desarrollo curricular, hemos aprendido a ejecutar código de forma lineal y a bifurcar los caminos del software basándonos en criterios de verdad. Sin embargo, en el desarrollo real de software, un Técnico se encuentra constantemente ante el requerimiento de repetir un conjunto de instrucciones un número determinado de veces, o hasta que se cumpla una condición específica. A estas estructuras se les denomina **estructuras repetitivas, ciclos o bucles**.

Imagina tener que calcular las notas finales de 35 estudiantes de Sistemas Informáticos, o procesar los consumos mensuales de agua de 500 familias de las zonas de San Antonio y San Expedito. Escribir el mismo código secuencial 500 veces sería ineficiente, aumentaría el tamaño del archivo innecesariamente y haría que el mantenimiento fuera imposible. Los bucles resuelven este problema permitiendo ejecutar el mismo bloque de código las veces que sea necesario, cambiando únicamente los datos que se procesan en cada vuelta o iteración.

Existen dos tipos principales de estructuras repetitivas que profundizaremos en el siguiente tema, pero que es fundamental mapear conceptualmente desde ahora:

1. **Ciclo con número definido de iteraciones (for):** Se utiliza cuando sabemos con exactitud, antes de iniciar la repetición, cuántas veces debe ejecutarse el código. Por ejemplo, recorrer la lista de los 10 barrios de Pailón para generar estadísticas comerciales individuales.
2. **Ciclo basado en condiciones (while):** Se ejecuta un número indefinido de veces mientras una condición lógica sea verdadera. El bucle se detiene en el momento exacto en que la condición pasa a ser falsa. Un ejemplo claro sería el cajero automático del Barrio Central Fátima: el sistema operativo sigue mostrando el menú de transacciones *mientras* el cliente no presione la opción de "Salir" o retirar su tarjeta.

Los bucles representan el verdadero músculo operativo de las computadoras, permitiéndoles procesar millones de datos por segundo sin fatiga, abriendo las puertas al análisis de datos a gran escala y a la automatización avanzada de procesos comerciales e industriales en nuestra sociedad.



3.6. OPTIMIZACIÓN DE CÓDIGO CONDICIONAL MEDIANTE OPERADORES TERNARIOS Y CORTOCIRCUITOS

En la práctica profesional del Técnico en Sistemas Informáticos, la eficiencia y la legibilidad del código son tan importantes como el correcto funcionamiento del algoritmo. Los lenguajes modernos ofrecen herramientas avanzadas para simplificar las estructuras condicionales tradicionales, reduciendo la cantidad de líneas escritas sin perder claridad conceptual.

Una de estas herramientas es el **operador ternario**, que permite condensar una estructura condicional doble (if / else) en una sola línea de código asignando un valor directo a una variable basado en el resultado de una condición. La sintaxis estándar en Python es: valor_si_verdadero if condicion else valor_si_falso.

Adicionalmente, la **evaluación en cortocircuito** aprovecha los operadores lógicos and y or para optimizar el rendimiento del procesador. Cuando el intérprete evalúa una condición compuesta con and, si la primera subcondición es falsa, el sistema ignora por completo la segunda subcondición porque sabe de antemano que la expresión general nunca podrá ser verdadera. Del mismo modo, en una estructura con or, si el primer término es verdadero, el cortocircuito aprueba la instrucción inmediatamente sin perder tiempo evaluando el resto de los componentes de la condición.

A continuación, apreciamos cómo aplicar estos métodos de optimización en un sistema de becas de estudio para los jóvenes de los barrios marginales de Pailón, como Las Misiones o Ovidio Barberi:

Python

```
# Sistema Optimizado de Clasificación de Estudiantes - Pailón Beneficios Sociocomunitarios
# Aplicación práctica de operadores ternarios y evaluación eficiente por cortocircuitopromedio_estudiante = 88.5 # Nota cuantitativa obtenida por el estudiante en el CEAsistencia_perfecta = True # Indicador booleano de disciplina y asistencia a Los Laboratorios
# 1. Implementación del operador ternario para definir la categoría en una sola línea
# Sintaxis compacta: Asigna "Excelente" si la nota es mayor a 85, de lo contrario asigna "Regular"
```



```
categoria = "
Excelente" if promedio_estudiante > 85.0 else "
Regular"
# 2. Demostración de evaluación en cortocircuito
# Si '
asistencia_perfecta
' fuera False, Python no gastaría recursos revisando el promedioif
asistencia_perfecta or promedio_estudiante > 90.0:    print(f"
El Técnico califica para incentivos institucionales. Condición: {
categoria}
.")
else:    print("
El expediente se archiva en la base de datos de seguimiento regular del Barrio
Saó.")
```

3.7. ESTRUCTURAS DE CONTROL FUNCIONALES Y DECLARATIVAS EN EL DESARROLLO MODERNO

A medida que la ingeniería de software ha evolucionado, el paradigma imperativo tradicional (dar órdenes explícitas paso a paso sobre cómo hacer las cosas) ha cedido espacio a enfoques más estructurados y declarativos. En la actualidad, el desarrollo de aplicaciones web y software empresarial demanda el uso de estructuras de control que traten los flujos de datos de manera compacta y segura, evitando la manipulación directa de variables de control globales.

En lenguajes avanzados y en el desarrollo Full-Stack con Python o JavaScript, las funciones declarativas como `map()`, `filter()` y las comprensiones de listas reemplazan de manera eficiente a los condicionales y bucles primitivos cuando se trabaja con colecciones de datos. En lugar de escribir múltiples líneas para filtrar elementos de una lista, la programación moderna permite pasar una función lógica directamente a la colección, abstrayendo la complejidad interna y reduciendo sustancialmente los errores de lógica.

Observemos un ejemplo técnico donde un listado de comercios registrados en los barrios de Pailón (Oto Waver, San José, etc.) es filtrado para obtener únicamente aquellos que pertenecen al sector de servicios tecnológicos, una tarea común para un Técnico que gestiona la base de datos del municipio:

Python



```
# Filtrado Declarativo de Datos Comerciales de Pailón
# Uso de comprensiones de listas como estructura de control moderna y segura
# Matriz bidimensional simulada que almacena [Nombre del Negocio, Rubro,
Barrio]comercios_pailon = [    ["
Tienda Don Bosco", "
Abarrotes", "
Barrio Central Fátima"],    ["
EduConect Computación", "
Tecnología", "
Barrio Saó"],    ["
Farmacia San José", "
Salud", "
Barrio San José"],    ["
Soporte Técnico Pailón", "
Tecnología", "
Barrio 27 de Mayo"]]
# Estructura de control declarativa: Filtra la lista en una sola línea compacta
# Extrae el comercio solo SI el elemento de la posición 1 (Rubro) es exactamente
"
Tecnología"
comercios_tech = [negocio for negocio in comercios_pailon if negocio[1] == "
Tecnología"]
# Despliegue automatizado de los resultados filtradosprint("
Comercios tecnológicos identificados en el municipio:")
print(comercios_tech)
```

3.8. EL ROL DE LAS ESTRUCTURAS DE CONTROL EN LOS MODELOS DE INTELIGENCIA ARTIFICIAL Y TOMA DE DECISIONES AUTOMATIZADA

En la era contemporánea, existe la creencia errónea de que la Inteligencia Artificial (IA) ha reemplazado por completo las estructuras de control lógicas de la programación tradicional. La realidad científica es completamente opuesta: los algoritmos de Machine Learning y las redes neuronales profundas dependen intrínsecamente de billones de microestructuras de control condicionales ejecutadas a nivel de hardware y software para poder procesar la información.

En un árbol de decisión (uno de los modelos predictivos de Inteligencia Artificial más utilizados en la industria financiera y de salud), el algoritmo se entrena calculando de forma automatizada miles de condicionales de tipo if / else ordenados jerárquicamente según la ganancia de información. Cuando la IA predice si un cliente pagará un crédito o si un sensor industrial fallará, lo que ocurre internamente es una evaluación secuencial y anidada ultra veloz de variables físicas y lógicas.



Para un Técnico en Sistemas Informáticos, dominar las estructuras básicas de control no es solo el primer paso para programar aplicaciones sencillas, sino también el cimiento matemático y lógico sobre el cual se construyen, depuran y optimizan los motores de Inteligencia Artificial que hoy en día transforman la gestión de datos en todo el mundo.

CIERRE TEÓRICO OBLIGATORIO

□ *Mitos vs. Realidades*

Mito Tecnológico	Realidad Científica
Las estructuras condicionales switch / case son idénticas a encadenar múltiples comandos if / else, por lo que da igual cuál usar en cualquier escenario de desarrollo.	El comando switch / case (o match en Python) es sustancialmente más eficiente debido a que el compilador genera una tabla de salto en memoria, accediendo directamente al caso verdadero sin tener que evaluar secuencialmente cada una de las condiciones previas.
Un programa de software puede funcionar perfectamente y ser catalogado como profesional sin necesidad de respetar las reglas de indentación o espaciado, ya que el procesador solo lee texto plano.	Falso. En lenguajes de alto nivel como Python, la indentación es una regla sintáctica obligatoria que define los bloques lógicos de control. Una mala indentación rompe la jerarquía del algoritmo y genera un fallo de ejecución inmediato.

□ *Glosario de Términos*

1. **Flujo de Ejecución:** El orden secuencial específico en el cual una computadora procesa e implementa las instrucciones lógicas codificadas dentro de un script de software.
2. **Expresión Booleana:** Una declaración lógica o matemática que adopta obligatoriamente uno de dos valores posibles del álgebra de Boole: Verdadero (True) o Falso (False).
3. **Indentación:** Espaciado intencional colocado al inicio de una línea de código fuente, utilizado para delimitar de manera visual y sintáctica el alcance jerárquico de las estructuras de control.
4. **Bifurcación:** Punto exacto dentro de un algoritmo computacional en el cual el flujo de datos se divide en dos o más caminos divergentes y excluyentes a raíz del resultado de una condición.



5. **Iteración:** El acto de repetir una o más instrucciones de código dentro de una estructura repetitiva, haciendo alusión a cada una de las vueltas o ciclos completados de manera exitosa.

□ **Ponte a Prueba**

1. Si un Técnico implementa una estructura condicional doble (if / else) en un programa de control de tránsito del Barrio 24 de Septiembre, ¿qué sucede si la expresión booleana evaluada resulta ser falsa?
 1. A) El software genera un error sintáctico severo y detiene la ejecución del procesador.
 2. B) Se ignora por completo el bloque dentro del if y se ejecuta obligatoriamente el bloque definido dentro del else.
 3. C) El programa salta de manera directa a la primera línea del script principal borrando la memoria caché.
2. ¿Cuál es la función principal del caso por defecto (representado por case _) dentro de una estructura condicional múltiple en el desarrollo moderno de aplicaciones?
 1. A) Crear un bucle infinito que impida el cierre de la terminal de comandos del sistema.
 2. B) Capturar cualquier valor o entrada imprevista que no coincida con ninguno de los casos específicos anteriores, garantizando la estabilidad del software.
 3. C) Borrar las variables secuenciales que fueron calculadas previamente en el inicio del script técnico.
3. ¿Qué regla crítica diferencia a las estructuras secuenciales de las estructuras condicionales en informática?
 1. A) Las secuenciales solo aceptan variables de tipo texto plano y prohíben las matemáticas.
 2. B) En las secuenciales, cada línea de código se ejecuta obligatoriamente una sola vez en un orden lineal estricto, mientras que las condicionales abren caminos excluyentes según criterios lógicos.



3. C) Las condicionales son exclusivas para entornos web y no se pueden ejecutar en consolas locales.

(Respuestas correctas: 1=B, 2=B, 3=B)



VALORACIÓN



El desarrollo e implementación de estructuras de control lógicas en el software dota a las computadoras de una capacidad crítica: la autonomía en la toma de decisiones. Como Técnicos en Sistemas Informáticos, esta capacidad nos exige un análisis profundo desde las dimensiones ética, social y medioambiental. Las decisiones automatizadas no son neutrales; responden directamente a los criterios, valores y rigurosidad con la que el programador diseña el código.

Desde el **impacto social y ético**, un algoritmo mal estructurado o carente de filtros de validación puede dar lugar a serios problemas de sesgo y discriminación. Imaginemos que el software diseñado para asignar los turnos de uso del coliseo en el Barrio 27 de Mayo o los cupos de laboratorio en el Barrio Saó favorece sistemáticamente a un sector por encima de otro debido a una condición mal planteada o una jerarquía injusta en el código anidado. La automatización de procesos tiene el objetivo fundamental de democratizar los servicios, aportar transparencia y agilizar la atención comunitaria, eliminando la discrecionalidad y el favoritismo humano, en lugar de amplificar las desigualdades.

En el ámbito de la **seguridad y la privacidad de la información**, la lógica de condiciones anidadas constituye la primera línea de defensa de los sistemas informáticos. Un fallo en la validación de credenciales del laboratorio del CEA Herman Ortiz Camargo podría dejar expuestos activos tecnológicos valiosos a accesos no autorizados. El Técnico debe programar asumiendo la responsabilidad de que cada if o match / case es un candado digital que protege identidades, registros académicos y la integridad física y lógica de la infraestructura pública y privada de su región.

Finalmente, con respecto al **impacto medioambiental**, el diseño óptimo de las estructuras de control se traduce en eficiencia energética. Un código mal optimizado, saturado de condicionales



innecesarios, anidamientos redundantes o bucles infinitos por error, obliga al procesador de la computadora a realizar millones de operaciones de cálculo inútiles por segundo. A gran escala, esto incrementa el consumo de energía eléctrica y acelera el desgaste del hardware, convirtiéndolo prematuramente en desecho electrónico (*e-waste*). El desarrollo de software sostenible exige escribir código limpio y eficiente, entendiendo que la elegancia lógica en la programación es también un acto de responsabilidad ecológica con nuestro entorno y nuestra comunidad.



PRODUCCIÓN



RETO FINAL / LABORATORIO PASO A PASO

Objetivo del Laboratorio: Diseñar de principio a fin un script ejecutable en Python que integre estructuras secuenciales, condicionales dobles, condicionales múltiples optimizadas y lógica de anidamiento para resolver un problema de automatización real en el municipio de Pailón.

Enunciado de la Necesidad Comunitaria

La Asociación de Productores y Comerciantes de Pailón, con sedes de acopio en el Barrio Central Fátima y en el Barrio San Antonio, requiere un sistema de facturación y despacho inteligente para organizar la distribución de suministros agrícolas e informáticos hacia los diferentes barrios de la población. El software debe recopilar los datos de la compra, clasificar el tipo de producto, aplicar descuentos automáticos basados en el volumen de compra y determinar la viabilidad del envío a domicilio gratuito en función del barrio de destino y el cumplimiento de las normativas de seguridad vial.

Guía Paso a Paso para el Técnico en Sistemas Informáticos

Paso 1: Configuración del Entorno de Desarrollo

1. Encienda la computadora en el laboratorio del CEA Herman Ortiz Camargo.
2. Abra su editor de código fuente preferido (Visual Studio Code o el entorno IDLE de Python).
3. Cree un archivo nuevo nombrado exactamente: `gestion_control_pailon.py`.



Paso 2: Desarrollo del Código Fuente

Escriba el siguiente programa, asegurándose de respetar la indentación de cuatro espacios y analizando minuciosamente cada comentario incorporado.

Python

[illegible]



```
---
---
---
---
---
---
---
---
---
---
---
---
---
---
---
---
---
---
---
---
---
---
---nombre_cliente = input("
Ingrese el nombre completo del solicitante: ")
barrio_destino = input("
Ingrese el barrio de destino en Pailón (ej. Sao, Fatima, Misiones):
").strip().lower()
print("\n
--- CATÁLOGO DE SUMINISTROS DISPONIBLES -
--")
print("1. Insumos Agrícolas / Semillas")
print("2. Equipos Tecnológicos / Computación")
print("3. Herramientas de Construcción General")
tipo_producto = input("
Seleccione la categoría del producto adquirido (1-3): ")
cantidad_unidades = int(input("
Ingrese la cantidad total de unidades compradas: "))
precio_unitario = float(input("
Ingrese el precio unitario del artículo (Bs.): "))
# Procesamiento Secuencial Básicosubtotal_compra = cantidad_unidades *
precio_unitario
# -
---
---
---
---
---
---
---
---
---
---
```



```
---  
---  
---  
---  
---  
---  
---  
---  
---  
---  
---  
---  
---  
---  
---  
---
```

```
# FASE 2: ESTRUCTURA CONDICIONAL MÚLTIPLE (Asignación de Descuentos por Rubro)  
# -
```

```
---  
---  
---  
---  
---  
---  
---  
---  
---  
---  
---  
---  
---  
---  
---  
---  
---  
---  
---  
---  
---  
---  
---  
---  
---
```

```
---porcentaje_descuento = 0.0 # Variable de control para almacenar el  
beneficiomatch tipo_producto: case "1": # Rubro Agrícola: Se incentiva  
al productor de Pailón if cantidad_unidades > 10:  
porcentaje_descuento = 0.10 # 10% de descuento por volumen de semillas case  
"2": # Rubro Tecnológico: Equipamiento para oficinas o estudiantes  
if subtotal_compra > 1000: porcentaje_descuento = 0.15 # 15% de  
descuento para promover el desarrollo tecnológico case "3": # Rubro  
Construcción: Material para los nuevos módulos vecinales if  
cantidad_unidades > 5: porcentaje_descuento = 0.05 # 5% de descuento
```

[illegible]



```

---
---
---
---
---
---
---costo_envio = 0.0envio_disponible = Trueprint("\n
--- PROCESANDO VIABILIDAD DE TRANSPORTE -
--")
# Primer nivel: Validación de zonas geográficas con coberturaif barrio_destino
== "
sao" or barrio_destino == "
fatima" or barrio_destino == "27 de mayo":    print(f"
Zona de Cobertura Confirmada: {
barrio_destino.capitalize()}
se encuentra en el radio urbano.")          # Segundo nivel anidado: Evaluación
económica para la gratuidad del servicio    if total_con_descuento >= 500.0:
print("
Criterio Económico Cumplido: Envío a domicilio completamente GRATUITO.")
costo_envio = 0.0    else:        print("
Monto de compra menor al umbral mínimo. Se aplica recargo logístico base.")
costo_envio = 20.0 # Costo de transporte expresado en Bolivianos        elif
barrio_destino == "
misiones" or barrio_destino == "
san expedito":    print(f"
Zona de Cobertura Extendida: {
barrio_destino.capitalize()}
requiere transporte pesado.")                # Segundo nivel anidado para zonas alejadas
if cantidad_unidades > 20:        print("
Envío autorizado con flota camiones de la estación Barrio 24 de Septiembre.")
costo_envio = 35.0    else:        print("
Envío denegado temporalmente por falta de cupo de carga ligera hacia esa ruta.")
envio_disponible = Falseelse:    print(f"
Error Logístico: El barrio '{
barrio_destino}
' no cuenta con rutas seguras consolidadas.")    envio_disponible = False
# -
---
---
---
---
---
---
---
---
---
---
---

```




```
print("
Costo de Transporte : Bs.", costo_envio)
if envio_disponible:    print("
TOTAL NETO A PAGAR : Bs.", total_facturado_final)    print("
ESTADO DEL TRÁMITE : FACTURADO Y EN RUTA DE ENTREGA")
else:    print("
TOTAL NETO A PAGAR : Bs.", total_con_descuento)    print("
ESTADO DEL TRÁMITE : RETENIDO - EL CLIENTE DEBE RECOGER EN EL SEDE CENTRAL")
print("=====")
```

Paso 3: Ejecución y Pruebas Lógicas en el Laboratorio

Corra el script utilizando el comando `python gestion_control_pailon.py` en la terminal e introduzca los siguientes tres casos de prueba para verificar de manera analítica que los condicionales anidados y múltiples responden exactamente según el flujo de diseño:

1. **Caso de Prueba A (Filtro Urbano Gratuito):** Cliente "Juan Pérez", Barrio "sao", Categoría "2" (Tecnología), Cantidad "2", Precio Unitario "600". *Resultado esperado:* Subtotal 1200 Bs., Descuento 15% (180 Bs.), Envío Gratis (0 Bs.), Total 1020 Bs. Estado: EN RUTA.
2. **Caso de Prueba B (Filtro Extendido Retenido):** Cliente "María Choque", Barrio "misiones", Categoría "1" (Agrícola), Cantidad "5", Precio Unitario "40". *Resultado esperado:* Subtotal 200 Bs., Descuento 0 Bs. Envío disponible: Falso. Estado: RETENIDO.
3. **Caso de Prueba C (Barrio Erróneo):** Cliente "Carlos Sosa", Barrio "centro", Categoría "3", Cantidad "1", Precio Unitario "10". *Resultado esperado:* Detecta barrio fuera de cobertura de forma segura, cancelando la logística automática.

RECURSOS ADICIONALES

1. **Documentación Oficial de Python - Estructuras de Control de Flujo:** Guía técnica detallada sobre el uso de condicionales, indentación y la sintaxis avanzada de comparación de patrones (match / case). Disponible en el portal oficial de documentación de Python.
2. **MDN Web Docs - Control de Flujo y Manejo de Errores:** Manual académico esencial enfocado en la lógica computacional de las estructuras de control condicionales y secuenciales aplicables al desarrollo de software y aplicaciones web integradas.



3. **Repositorio Comunitario de Algoritmia Básica en GitHub:** Colección de código abierto que contiene problemas lógicos resueltos, diagramas de flujo interactivos y laboratorios prácticos diseñados para estudiantes e instructores del área de Sistemas Informáticos.